



TID Complete Source Code

The Instant Destroyer — Linux Kernel Module Compiler-Resistant Secure Memory & Cache Zeroization

Author	Ahmad Qasim Mohammad Hassan
ORCID	0009-0001-4360-0802
Version	1.0 — Final Production Release
License	AGPL-3.0
Target	x86_64 — Linux Kernel Module (Ring 0)
Files	tid_uapi.h tid.c Makefile tid_user.c

⚙️ Project Architecture

tid_uapi.h → Shared ABI between kernel and user space
tid.c → Kernel module — Ring 0 — CLFLUSHOPT + REP STOSQ + MFENCE
Makefile → Build and load the kernel module
tid_user.c → Command-line tool to invoke TID via /dev/tid

1 Shared Header File

tid_uapi.h

Defines the shared ABI between kernel and user space — the data structure and ioctl command. Alignment is fixed at 32 bytes to guarantee strict binary compatibility.

```
#ifndef _TID_UAPI_H
#define _TID_UAPI_H

#include <linux/types.h>
#include <linux/ioctl.h>

/*
 * Data structure exchanged between user space and the kernel.
 * ABI alignment fixed at 32 bytes for strict binary compatibility.
 */
struct tid_args {
    __u64 size;           /* size of region to wipe (bytes) */
    __u64 user_addr;      /* user-space address of the region */
    __u64 elapsed_ns;      /* execution time in nanoseconds (output) */
    __u32 success_code;    /* 0x5449444F on success (TIDO) */
    __u32 _pad;           /* ABI padding */
};

#define TID_MAGIC 'T'
#define TID_IOCTL_ZERO _IOWR(TID_MAGIC, 1, struct tid_args)

#define TID_SUCCESS_CODE 0x5449444Fu

#endif /* _TID_UAPI_H */
```

2 Kernel Module

tid.c

```
// SPDX-License-Identifier: AGPL-3.0-only
/*
 * This file is licensed under AGPL-3.0-only.
 * MODULE_LICENSE("GPL") is required by the Linux kernel
 * infrastructure and does not override the AGPL-3.0 license.
 */
/*
 * TID - The Instant Destroyer
 * Linux kernel module for compiler-resistant secure memory
 * and cache zeroization on x86_64.
 *
 * Author : Ahmad Qasim Mohammad Hassan
 * ORCID  : 0009-0001-4360-0802
 */
```

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/miscdevice.h>
#include <linux/uaccess.h>
#include <linux/mm.h>
#include <linux/highmem.h>
#include <linux/sched/mm.h>
#include <linux/ktime.h>
#include <linux/slab.h>
#include <asm/cpufeature.h>

#include "tid_uapi.h"

#define TID_DEVICE_NAME "tid"
#define TID_MAX_SIZE (16UL * 1024UL * 1024UL) /* 16 MiB upper limit */
#define TID_CACHE_LINE 64UL
```

Core Computation | tid_zeroize_core()

Three sequential phases: REP STOSQ → CLFLUSHOPT → MFENCE. Runs in Ring 0, compiler-optimization-resistant thanks to `noinline` + `asm volatile`.

```
/* -----
 * Core routine: three sequential phases
 * 1) Compiler-resistant wipe using REP STOSQ
 * 2) Cache flush CLFLUSHOPT over the entire original range
 * 3) Full memory barrier MFENCE
 * ----- */
static noinline void tid_zeroize_core(void *buf, size_t size)
{
    void          *flush_base = buf; /* reference start address */
    void          *addr       = buf;
    size_t        qwords      = size >> 3;
    size_t        remainder    = size & 0x7UL;
    uintptr_t     start, end, line;

    /* Phase 1: wipe qwords then remainder */
```

```

    if (qwords) {
        asm volatile ("rep stosq"
                      : "+D"(addr), "+c"(qwords)
                      : "a"(0ULL)
                      : "memory");
    }
    if (remainder) {
        u8 *p = (u8 *)addr;
        while (remainder--)
            *p++ = 0;
    }

    /* Phase 2: flush cache over the entire original range */
    start = (uintptr_t)flush_base & ~(TID_CACHE_LINE - 1UL);
    end   = ((uintptr_t)flush_base + size + TID_CACHE_LINE - 1UL)
           & ~(TID_CACHE_LINE - 1UL);

    for (line = start; line < end; line += TID_CACHE_LINE)
        asm volatile ("clflushopt (%0)"
                      :: "r"((void *)line)
                      : "memory");

    /* Phase 3: full memory barrier */
    asm volatile ("mfence" ::: "memory");
}

```

Self-Test | tid_self_test()

Runs automatically on module load — tests 25 different sizes from 1 byte up to 64 kilobytes.

```

/* -----
 * Self-test on load: verifies functional correctness on the target hardware
 * ----- */
static int __init tid_self_test(void)
{
    static const size_t sizes[] = {
        1, 7, 8, 9, 15, 16, 17, 31, 32, 33,
        37, 63, 64, 65, 100, 127, 128, 256, 511,
        512, 1024, 4095, 4096, 65535, 65536
    };
    size_t i, j;
    u8 *buf;

    for (i = 0; i < ARRAY_SIZE(sizes); i++) {
        buf = kmalloc(sizes[i], GFP_KERNEL);
        if (!buf)
            return -ENOMEM;
        memset(buf, 0xAB, sizes[i]);

        tid_zeroize_core(buf, sizes[i]);

        for (j = 0; j < sizes[i]; j++) {
            if (buf[j] != 0) {
                pr_err("TID: self-test FAILED size=%zu byte[%zu]=0x%02x\n",
                      sizes[i], j, buf[j]);
                kfree(buf);
                return -EINVAL;
            }
        }
        kfree(buf);
    }
    pr_info("TID: self-test PASSED (%zu sizes)\n", ARRAY_SIZE(sizes));
}

```

```

    return 0;
}

```

ioctl Handler | tid_ioctl()

Receives requests from user space, pins memory pages with `pin_user_pages_fast`, then executes the secure wipe.

```

/* -----
 * ioctl handler: pins user pages and executes the wipe
 * ----- */
static long tid_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct tid_args  uargs;
    struct page      **pages = NULL;
    void             *kbuf   = NULL;
    unsigned long     npages, off, first_page;
    long              pinned = 0;
    ktime_t           t0, t1;
    long              ret = 0;

    if (cmd != TID_IOCTL_ZERO)
        return -ENOTTY;
    if (copy_from_user(&uargs, (void __user *)arg, sizeof(uargs)))
        return -EFAULT;
    if (uargs.size == 0 || uargs.size > TID_MAX_SIZE)
        return -EINVAL;

    first_page = uargs.user_addr & PAGE_MASK;
    off         = uargs.user_addr & (PAGE_SIZE - 1);
    npages      = (off + uargs.size + PAGE_SIZE - 1) >> PAGE_SHIFT;

    pages = kmalloc_array(npages, sizeof(*pages), GFP_KERNEL);
    if (!pages)
        return -ENOMEM;

    pinned = pin_user_pages_fast(first_page, npages,
                                FOLL_WRITE | FOLL_LONGTERM, pages);

    if (pinned != (long)npages) {
        ret = (pinned < 0) ? pinned : -EFAULT;
        goto out_unpin;
    }

    /* map pages into a temporary kernel buffer */
    kbuf = vmmap(pages, npages, VM_MAP, PAGE_KERNEL);
    if (!kbuf) {
        ret = -ENOMEM;
        goto out_unpin;
    }

    t0 = ktime_get();
    tid_zeroize_core((u8 *)kbuf + off, uargs.size);
    t1 = ktime_get();

    vunmap(kbuf);

    uargs.elapsed_ns = ktime_to_ns(ktime_sub(t1, t0));
    uargs.success_code = TID_SUCCESS_CODE;

    if (copy_to_user((void __user *)arg, &uargs, sizeof(uargs)))
        ret = -EFAULT;

out_unpin:

```

```

    if (pinned > 0)
        unpin_user_pages(pages, pinned);
    kfree(pages);
    return ret;
}

```

Device Registration | file_operations + module_init

```

/* -----
 * Device registration
 * ----- */
static const struct file_operations tid_fops = {
    .owner          = THIS_MODULE,
    .unlocked_ioctl = tid_ioctl,
    .compat_ioctl   = tid_ioctl,
};

static struct miscdevice tid_misc = {
    .minor = MISC_DYNAMIC_MINOR,
    .name  = TID_DEVICE_NAME,
    .fops  = &tid_fops,
    .mode  = 0600,
};

static int __init tid_init(void)
{
    int rc;

    if (!boot_cpu_has(X86_FEATURE_CLFLUSHOPT)) {
        pr_err("TID: CPU lacks CLFLUSHOPT support\n");
        return -ENODEV;
    }

    rc = tid_self_test();
    if (rc)
        return rc;

    rc = misc_register(&tid_misc);
    if (rc) {
        pr_err("TID: misc_register failed (%d)\n", rc);
        return rc;
    }
    pr_info("TID: loaded (/dev/%s)\n", TID_DEVICE_NAME);
    return 0;
}

static void __exit tid_exit(void)
{
    misc_deregister(&tid_misc);
    pr_info("TID: unloaded\n");
}

module_init(tid_init);
module_exit(tid_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Ahmad Qasim Mohammad Hassan <ORCID: 0009-0001-4360-0802>");
MODULE_DESCRIPTION("The Instant Destroyer: secure memory + cache zeroization");
MODULE_VERSION("1.0");

```

3 Build File

Makefile

Builds the kernel module with -O2 and provides load/unload commands.

```
obj-m += tid.o

KDIR ?= /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

EXTRA_CFLAGS += -O2 -Wall -Wextra

all:
$(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
$(MAKE) -C $(KDIR) M=$(PWD) clean

install:
sudo insmod tid.ko

uninstall:
sudo rmmod tid
```

4 User-Space Tool

tid_user.c

Simple command-line tool to invoke TID via /dev/tid — verifies correctness byte by byte.

```
/* Simple command-line tool to invoke TID on a user-space buffer */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include "tid_uapi.h"

int main(int argc, char **argv)
{
    size_t      sz;
    void        *buf;
    int         fd;
    struct tid_args a = {0};

    if (argc != 2) {
        fprintf(stderr, "usage: %s <size_bytes>\n", argv[0]);
        return 2;
    }
    sz = strtoul(argv[1], NULL, 0);

    buf = aligned_alloc(64, (sz + 63) & ~63UL);
    if (!buf) { perror("alloc"); return 1; }
    memset(buf, 0xAB, sz);
```

```

fd = open("/dev/tid", O_RDWR);
if (fd < 0) { perror("open /dev/tid"); free(buf); return 1; }

a.size      = sz;
a.user_addr = (unsigned long)buf;

if (ioctl(fd, TID_IOCTL_ZERO, &a) < 0) {
    perror("ioctl");
    close(fd); free(buf); return 1;
}
close(fd);

if (a.success_code != TID_SUCCESS_CODE) {
    fprintf(stderr, "bad success_code: 0x%08x\n", a.success_code);
    free(buf); return 1;
}

/* byte-by-byte verification */
for (size_t i = 0; i < sz; i++) {
    if (((unsigned char *)buf)[i] != 0) {
        fprintf(stderr, "byte %zu = 0x%02x (NOT zero)\n",
            i, ((unsigned char *)buf)[i]);
        free(buf); return 1;
    }
}

printf("OK size=%zu elapsed=%llu ns\n",
    sz, (unsigned long long)a.elapsed_ns);
free(buf);
return 0;
}

```

⚙️ Build & Load Commands

```

# 1. Build the kernel module
make

# 2. Verify clflushopt is present in the binary
objdump -d tid.ko | grep clflushopt

# 3. Load the module (runs self-test automatically)
sudo insmod tid.ko
dmesg | tail -3

# 4. Test from user space (256 bytes)
gcc -O2 -o tid_user tid_user.c
./tid_user 256
# Expected: OK size=256 elapsed=... ns

# 5. Unload the module
sudo rmmod tid

```

— End of Code —

Ahmad Qasim Mohammad Hassan | ORCID: 0009-0001-4360-0802 | TID Project — 2026